

Cognifyz Data Science Internship Tasks

This notebook covers Level 1, Level 2, and Level 3 tasks.

```
In [1]: # =====
# Import Required Libraries
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Show plots inline
%matplotlib inline
```

```
In [2]: # =====
# Load Dataset
# =====
# Replace 'Dataset .csv' with your dataset file
df = pd.read_csv("Dataset .csv")
df.head()
```

Out[2]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose	Longitude	Latitude	Cuisines	...	Curr
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...	121.027535	14.565443	French, Japanese, Desserts	...	Bots Pt
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...	121.014101	14.553708	Japanese	...	Bots Pt
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...	121.056831	14.581404	Seafood, Asian, Filipino, Indian	...	Bots Pt
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	121.056475	14.585318	Japanese, Sushi	...	Bots Pt
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	121.057508	14.584450	Japanese, Korean	...	Bots Pt

5 rows × 21 columns

Level 1 Tasks: Data Exploration & Descriptive Analysis

```
In [3]: # Task 1: Data Exploration and Preprocessing
print("Shape of dataset:", df.shape)
print("Missing values per column:\n", df.isnull().sum())
df.info()
```

```

Shape of dataset: (9551, 21)
Missing values per column:
Restaurant ID      0
Restaurant Name    0
Country Code       0
City               0
Address            0
Locality           0
Locality Verbose   0
Longitude          0
Latitude           0
Cuisines           9
Average Cost for two 0
Currency           0
Has Table booking  0
Has Online delivery 0
Is delivering now   0
Switch to order menu 0
Price range        0
Aggregate rating    0
Rating color        0
Rating text         0
Votes              0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Restaurant ID          9551 non-null  int64
1   Restaurant Name        9551 non-null  object
2   Country Code           9551 non-null  int64
3   City                   9551 non-null  object
4   Address                9551 non-null  object
5   Locality               9551 non-null  object
6   Locality Verbose       9551 non-null  object
7   Longitude              9551 non-null  float64
8   Latitude               9551 non-null  float64
9   Cuisines               9542 non-null  object
10  Average Cost for two    9551 non-null  int64
11  Currency               9551 non-null  object
12  Has Table booking       9551 non-null  object
13  Has Online delivery     9551 non-null  object
14  Is delivering now       9551 non-null  object
15  Switch to order menu    9551 non-null  object
16  Price range            9551 non-null  int64
17  Aggregate rating        9551 non-null  float64
18  Rating color           9551 non-null  object
19  Rating text            9551 non-null  object
20  Votes                  9551 non-null  int64
dtypes: float64(3), int64(5), object(13)
memory usage: 1.5+ MB

```

```

In [4]: # Handle missing values (basic strategy)
for col in df.select_dtypes(include=["float64", "int64"]).columns:
    df[col].fillna(df[col].mean(), inplace=True)

for col in df.select_dtypes(include=["object"]).columns:
    df[col].fillna(df[col].mode()[0], inplace=True)

```

C:\Users\drman\AppData\Local\Temp\ipykernel_22472\1897701415.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].mean(), inplace=True)
```

C:\Users\drman\AppData\Local\Temp\ipykernel_22472\1897701415.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].mode()[0], inplace=True)
```

```

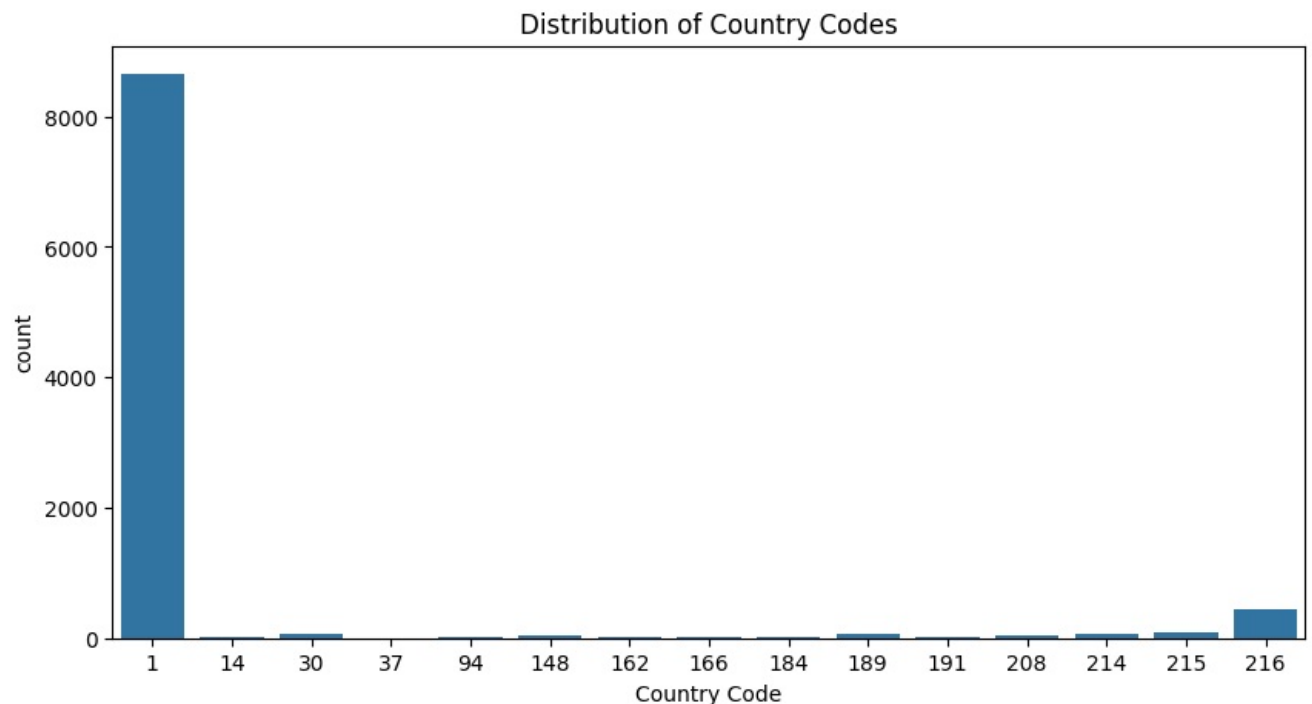
In [5]: # Task 2: Descriptive Analysis
print(df.describe())

```

```
if "Country Code" in df.columns:
    plt.figure(figsize=(10,5))
    sns.countplot(data=df, x="Country Code")
    plt.title("Distribution of Country Codes")
    plt.show()
```

	Restaurant ID	Country Code	Longitude	Latitude \
count	9.551000e+03	9551.000000	9551.000000	9551.000000
mean	9.051128e+06	18.365616	64.126574	25.854381
std	8.791521e+06	56.750546	41.467058	11.007935
min	5.300000e+01	1.000000	-157.948486	-41.330428
25%	3.019625e+05	1.000000	77.081343	28.478713
50%	6.004089e+06	1.000000	77.191964	28.570469
75%	1.835229e+07	1.000000	77.282006	28.642758
max	1.850065e+07	216.000000	174.832089	55.976980

	Average Cost for two	Price range	Aggregate rating	Votes
count	9551.000000	9551.000000	9551.000000	9551.000000
mean	1199.210763	1.804837	2.666370	156.909748
std	16121.183073	0.905609	1.516378	430.169145
min	0.000000	1.000000	0.000000	0.000000
25%	250.000000	1.000000	2.500000	5.000000
50%	400.000000	2.000000	3.200000	31.000000
75%	700.000000	2.000000	3.700000	131.000000
max	800000.000000	4.000000	4.900000	10934.000000



Level 2 Tasks: Business Analysis & Feature Engineering

```
In [6]: # Task 1: Table Booking and Online Delivery
if "Has Table booking" in df.columns:
    print("Table booking availability (%):")
    print(df["Has Table booking"].value_counts(normalize=True) * 100)

if "Has Online delivery" in df.columns:
    print("\nOnline delivery availability (%):")
    print(df["Has Online delivery"].value_counts(normalize=True) * 100)
```

Table booking availability (%):

```
Has Table booking
No      87.875615
Yes     12.124385
Name: proportion, dtype: float64
```

Online delivery availability (%):

```
Has Online delivery
No      74.337766
Yes     25.662234
Name: proportion, dtype: float64
```

```
In [7]: # Task 2: Price Range Analysis
if "Price range" in df.columns and "Aggregate rating" in df.columns:
    print(df["Price range"].value_counts())
    avg_ratings = df.groupby("Price range")["Aggregate rating"].mean()
    print(avg_ratings)
```

```
avg_ratings.plot(kind="bar", title="Average Ratings by Price Range")
plt.show()
```

Price range

```
1    4444
2    3113
3    1408
4     586
```

Name: count, dtype: int64

Price range

```
1    1.999887
2    2.941054
3    3.683381
4    3.817918
```

Name: Aggregate rating, dtype: float64



```
In [8]: # Task 3: Feature Engineering Example
if "Restaurant Name" in df.columns:
    df["Restaurant Name Length"] = df["Restaurant Name"].apply(lambda x: len(str(x)))

if "Has Table booking" in df.columns:
    df["Has Table Booking (Encoded)"] = df["Has Table booking"].map({"Yes":1, "No":0})

df.head()
```

Out[8]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose	Longitude	Latitude	Cuisines	...	On deliv
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...	121.027535	14.565443	French, Japanese, Desserts	...	
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...	121.014101	14.553708	Japanese	...	
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...	121.056831	14.581404	Seafood, Asian, Filipino, Indian	...	
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	121.056475	14.585318	Japanese, Sushi	...	
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	121.057508	14.584450	Japanese, Korean	...	

5 rows × 23 columns

Level 3 Tasks: Predictive Modeling & Visualization

In [9]:

```
# Task 1: Predictive Modeling (Regression)
if "Aggregate rating" in df.columns:
    df_encoded = pd.get_dummies(df, drop_first=True)

    features = df_encoded.drop(columns=["Aggregate rating"], errors="ignore")
    target = df_encoded["Aggregate rating"]

    X_train, X_test, y_train, y_test = train_test_split(
        features, target, test_size=0.2, random_state=42
    )

    models = {
        "Linear Regression": LinearRegression(),
        "Decision Tree": DecisionTreeRegressor(),
        "Random Forest": RandomForestRegressor()
    }

    for name, model in models.items():
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        print(name)
        print("R2 Score:", r2_score(y_test, preds))
        print("MSE:", mean_squared_error(y_test, preds))
        print("-" * 40)

Linear Regression
R2 Score: 0.9596117077449599
MSE: 0.09192815510494368
-----
Decision Tree
R2 Score: 0.9766624593979982
MSE: 0.053118785975928826
-----
Random Forest
R2 Score: 0.9881382431130001
MSE: 0.02699865149136572
-----
```

```
In [10]: # Task 2 & 3: Customer Preference Analysis & Data Visualization
if "Cuisines" in df.columns and "Aggregate rating" in df.columns:
    top_cuisines = df["Cuisines"].value_counts().head(10).index
    plt.figure(figsize=(12,6))
    sns.barplot(data=df[df["Cuisines"].isin(top_cuisines)], x="Cuisines", y="Aggregate rating")
    plt.title("Average Ratings by Top 10 Cuisines")
    plt.xticks(rotation=45)
    plt.show()

if "Aggregate rating" in df.columns:
    plt.figure(figsize=(10,5))
    sns.histplot(df["Aggregate rating"], bins=20, kde=True)
    plt.title("Distribution of Ratings")
    plt.show()
```

